

NORMA 220501095

# Documento de diseño de software

Cooperativa Financiera El Progreso

Sistema de gestión de asociados, movimientos y reportes administrativos

Versión 1.0 | 2026

# 1. Introducción

Cooperativa El Progreso es una aplicación de consola diseñada para apoyar la operación de una cooperativa financiera. Permite al cajero registrar y consultar asociados, registrar consignaciones y retiros, consultar movimientos y convertir el saldo a dólares usando la TRM oficial. También ofrece al administrador reportes para analizar la actividad de la cooperativa.

El sistema representa una cuenta de ahorro por asociado. El saldo no se almacena como un valor editable: se calcula sumando los movimientos aceptados. Esta decisión evita inconsistencias y conserva el historial de operaciones.

# 2. Alcance

- Gestión de asociados: registrar, consultar, actualizar y eliminar bajo las reglas definidas.
- Gestión de movimientos: consignaciones, retiros, comisiones y validación de saldo.
- Consulta de TRM mediante un servicio externo oficial.
- Seis reportes administrativos.
- Interfaz de consola para los roles de cajero y administrador.

**Alcance actual:** la aplicación utiliza almacenamiento en memoria y no incluye una interfaz web ni una base de datos persistente. El modelo de base de datos incluido en esta evidencia es una propuesta para una versión futura.

# 3. Requisitos funcionales

| ID    | Requisito                                                                   |
|-------|-----------------------------------------------------------------------------|
| RF-01 | Registrar un asociado con documento, nombre y datos de contacto.            |
| RF-02 | Rechazar documentos duplicados.                                             |
| RF-03 | Buscar asociados por documento o nombre.                                    |
| RF-04 | Registrar consignaciones de valor positivo.                                 |
| RF-05 | Registrar retiros sin permitir saldos negativos.                            |
| RF-06 | Aplicar una comisión de \$8.000 COP a retiros superiores a \$1.000.000 COP. |
| RF-07 | Consultar el saldo calculado a partir de los movimientos.                   |
| RF-08 | Convertir el saldo a dólares con la TRM oficial.                            |
| RF-09 | Consultar reportes administrativos.                                         |

## 4. Requisitos no funcionales

---

- **Disponibilidad:** un fallo de la API de TRM no debe cerrar la aplicación.
- **Mantenibilidad:** las responsabilidades se separan por capas.
- **Pruebas:** la lógica principal se verifica con pruebas unitarias xUnit.
- **Integridad:** los movimientos aceptados son de solo adición y no se borran.
- **Tecnología:** C# 14 y .NET 10.

## 5. Arquitectura del software

---

Se utiliza una arquitectura por capas dentro de un proyecto ejecutable:

| Capa           | Responsabilidad                                                              |
|----------------|------------------------------------------------------------------------------|
| UI             | Presenta menús de cajero y administrador y recoge entradas del usuario.      |
| Services       | Contiene las reglas de negocio y coordina repositorios y servicios externos. |
| Repositories   | Abstrae el acceso al almacenamiento de asociados y movimientos.              |
| Data           | Proporciona el almacenamiento en memoria compartido.                         |
| Infrastructure | Integra la API externa de TRM.                                               |
| Models         | Representa entidades, resultados y DTO de reportes.                          |

El punto de composición es `Program.cs`, donde se crean las dependencias y se inyectan en los servicios y menús.

## 6. Descripción de módulos

---

| Módulo              | Contenido                                                                          |
|---------------------|------------------------------------------------------------------------------------|
| Asociados           | <code>MemberService</code> y <code>MemberRepository</code> .                       |
| Movimientos         | <code>MovementService</code> y <code>MovementRepository</code> .                   |
| Reportes            | <code>ReportService</code> y modelos bajo <code>Models/Reports</code> .            |
| TRM                 | <code>TrmExchangeRateService</code> e interfaz <code>IExchangeRateService</code> . |
| Interfaz de usuario | <code>CashierMenu</code> y <code>ManagerMenu</code> .                              |

## 7. Justificación técnica

---

- Las interfaces reducen el acoplamiento y permiten reemplazar implementaciones.

- El saldo derivado evita que se pueda editar manualmente.
- `OperationResult<T>` comunica rechazos esperados sin usar excepciones para cada regla de negocio.
- La API está aislada en infraestructura porque es una integración externa, no una regla del negocio.
- Las pruebas pueden usar `FakeExchangeRateService` y no dependen de Internet.

## 8. Conclusión

---

El diseño prioriza separación de responsabilidades, trazabilidad de movimientos y tolerancia ante fallos externos. La propuesta permite evolucionar desde el almacenamiento en memoria hacia una base de datos sin modificar la lógica de negocio principal.